

CODEX VERIFICATION AGENT FOR RTL TESTBENCH GENERATION
MINI PROJECT 2 ? PHASE 2
TOPIC 2: AI FOR DESIGN VERIFICATION

This project implements a Codex-based verification agent that reads a natural-language RTL specification and a set of candidate Verilog implementations, generates a self-checking Verilog testbench, and evaluates that testbench through simulation. The goal is to automatically isolate the single correct RTL implementation among buggy mutants.

1. METHODOLOGY & PROBLEM DEFINITION

We address the Google verification-style ICLAD task. Each problem contains:

- ? a natural-language specification (specification.md)
- ? multiple candidate RTL implementations (mutant_*.v)

Only one RTL implementation is correct. The agent must generate a Verilog testbench that passes only the correct design.

The scoring metric is:

Score = $1 / (\text{number of passing implementations})$

If the correct implementation fails the testbench, the score is 0. The ideal result is 1.0, meaning exactly one implementation passes.

Our methodology is a structured generate-evaluate-refine workflow:

1. parse the specification into structured JSON
2. build a test plan
3. generate a self-checking Verilog testbench
4. run simulation on all candidate RTL implementations
5. refine if the result is not discriminating enough

This approach is suitable because the task requires behavioral precision, not just syntactically valid code generation.

2. AGENT ARCHITECTURE (COMPONENTS, DESIGN, DIFFERENCES FROM PROMPTING)

The repository is organized as a concrete agent workspace:

```
verification-agent/  
??? AGENTS.md  
??? agent.py  
??? run_pipeline.py  
??? run_sim.py  
??? spec_parser.py  
??? tb_generator.py  
??? simulator.py  
??? prompts/  
??? sample_problem/  
??? tb/  
??? artifacts/
```

Main components:

- ? agent.py: orchestrates parsing, planning, generation, simulation, and refinement

- ? run_pipeline.py: single entry-point script for the full workflow
- ? run_sim.py: standalone simulation entry point
- ? spec_parser.py: structured behavior extraction
- ? tb_generator.py: Verilog testbench generation
- ? simulator.py: iverilog / vvp execution and result collection

The agent loop is:

```
specification.md + mutant_*.v
?
  parse behavior
  ?
    generate test plan
    ?
      generate testbench
      ?
        simulate
        ?
          analyze results
          ?
            refine if needed
```

This differs from one-shot prompting because the system preserves intermediate artifacts, invokes external tools, reads execution results, and can improve later outputs based on failures.

3. TOOLS & FRAMEWORKS (EDA TOOLS, LLMS, AGENT FRAMEWORKS)

Tools used:

- ? Codex CLI: model execution interface
- ? gpt-5-codex: specification parsing, test-plan generation, and testbench generation
- ? iverilog: Verilog compilation
- ? vvp: simulation execution
- ? Python scripts: orchestration and result processing

We do not use a separate external agent framework. The agent logic is implemented directly in Python as an explicit planner-executor-feedback loop.

4. CONTEXT ENGINEERING (INPUTS, REPRESENTATION, CONTEXT MANAGEMENT)

The agent receives:

- ? raw specification text
- ? candidate RTL implementations
- ? recovered module header information
- ? simulation feedback from prior iterations

The context is represented as:

- ? raw text for the original specification
- ? structured JSON for parsed behavior
- ? structured JSON for the generated test plan
- ? JSON logs and score summaries for simulation results

To control context size, parsed behavior and test plans are summarized before being reused in large prompts. This reduced Codex timeouts and kept generation requests manageable.

5. INTERACTION WITH EDA TOOLS (INVOCATION, PARSING, ITERATION)

The agent uses Python subprocess calls to invoke:

- ? iverilog -g2012 to compile the generated testbench with each mutant RTL file
- ? vvp to execute the simulation

Simulation output is parsed using explicit PASS and FAIL markers. Results are stored as JSON under artifacts/. This tool interaction is part of an iterative loop: if the score is weak or the generated testbench fails, the agent can revise the next attempt.

6. PROTOTYPE IMPLEMENTATION (WHAT WORKS / INCOMPLETE PARTS)

Currently functional:

- ? natural-language specification parsing
- ? structured spec and test-plan artifact generation
- ? self-checking Verilog testbench generation
- ? automated simulation across all candidates in a problem directory
- ? score computation and JSON result reporting
- ? single-command execution via run_pipeline.py

Still incomplete:

- ? full visible-benchmark evaluation
- ? systematic baseline comparison
- ? stronger robustness on more complex vectorized modules

7. INITIAL RESULTS (≥ 1 TESTCASE, METRICS, BASELINE IF AVAILABLE)

We evaluated the prototype on three real visible problems from the public ICLAD Google Verification benchmark, using one iteration per problem.

Problem	Mutants	Passing	Implementations	Compile Errors	Runtime (s)	Observation
enc_bin2gray	31	1	0	62.6	Strong discrimination	
enc_bin2onehot	31	1	0	141.8	Strong discrimination	
shift_right	31	0	31	234.4	Generated testbench failed to compile	

The first two results show that the agent can reduce 31 candidate implementations to a single passing design in one iteration. The shift_right failure exposes a real limitation: the generated testbench used variable part-select expressions that iverilog rejected.

We also verified the packaged sample_problem, where the agent achieves 1/6 passing with no compile errors.

8. AI USAGE & INSIGHTS (CRITICAL)

AI is central in three places:

- ? extracting structured hardware behavior from English specifications
- ? generating structured test plans
- ? generating and refining Verilog testbenches

Where AI works well:

- ? interpreting ambiguous natural-language behavior
- ? proposing useful directed and corner-case tests

? generating good first-pass testbenches for simpler combinational problems

Where AI is weaker:

- ? strict tool-compatibility details
- ? edge cases involving complex packed-vector indexing
- ? consistently generating simulator-safe Verilog for all interface styles

This means AI is effective for reasoning and candidate generation, but it still benefits from deterministic constraints and execution-based validation.

9. CHALLENGES, LIMITATIONS, IMPROVEMENTS

Key challenges:

- ? LLM timeouts on large prompts
- ? invisible progress during long generations
- ? generated testbenches missing timing-specific checks
- ? generated Verilog that compiles poorly on some interface patterns

Fixes implemented in this phase:

- ? configurable Codex timeout
- ? prompt compaction
- ? line-buffered logging
- ? deterministic synchronous-reset guard
- ? iverilog -g2012 compilation support

Main limitations:

- ? not yet evaluated on the full visible benchmark set
- ? not yet robust across all vectorized module shapes
- ? refinement still depends partly on prompt behavior

Phase 3 improvements:

- ? run the full visible benchmark set
- ? collect aggregate metrics
- ? strengthen deterministic testbench post-processing
- ? improve refinement on failing compile cases

10. CONCRETE OUTPUTS AND REPRODUCIBILITY

The agent produces explicit artifacts, including:

- ? generated testbenches under tb/
- ? parsed specifications under artifacts/<problem>/parsed_spec.json
- ? test plans under artifacts/<problem>/test_plan.json
- ? simulation summaries under artifacts/<problem>/iteration_1_results.json
- ? final results under artifacts/<problem>/final_result.json

The complete workflow is reproducible with one command:

```
python3 run_pipeline.py --problem-dir sample_problem --max-iters 1
```

Real benchmark result artifacts for enc_bin2gray, enc_bin2onehot, and shift_right are included in the submission package.

11. SUMMARY

This project implements a concrete Codex-based RTL verification agent that reads a natural-language specification, generates a self-checking Verilog testbench, runs simulation against candidate implementations, and improves the workflow through execution feedback. The Phase 2 prototype already shows strong early performance on real ICLAD visible problems, while also exposing a meaningful failure mode that defines the next engineering target for Phase 3.